

PROJECT REPORT

AI Email Copilot using Retrieval-Augmented Generation (RAG)

Application Name:	AI Email Copilot
Domain:	Artificial Intelligence Generative AI Cloud Computing
Submitted To:	IBM SkillsBuild / Project Evaluation Committee
Prepared By:	Team GEN CREW (Unique ID: IBMBH00696)
College Name:	AJAY KUMAR GARG ENGINEERING COLLEGE

1. Project Title

AI Email Copilot using Retrieval-Augmented Generation (RAG) and Google Gemini API.

2. Application Name

AI Email Copilot

3. Problem Statement

Writing professional emails is time-consuming and challenging for students, job seekers, and fresh professionals. Most existing AI generation tools rely solely on generic prompt engineering, producing contextually shallow emails. Without an explicit retrieval grounding mechanism, the model lacks alignment with professional templates and standard communication norms. This project implements RAG to dynamically fetch relevant email template examples before prompting the generator, ensuring tone-perfect structure.

4. Project Objectives

- Design and deploy an AI-powered web application that generates professional context-grounded emails.
- Retrieve relevant email templates from a local vector store using Google gemini-embedding-2 API.
- Leverage the Google Gemini 3.5 Flash LLM to produce structured, natural, and tone-accurate copy.
- Decouple the setup into a REST API backend hosted on AWS Elastic Beanstalk and a Streamlit Cloud UI frontend.

5. Proposed Solution

- Uses Retrieval-Augmented Generation (RAG) to fetch matching sample categories (e.g. cold email, internship request, apology).
- Employs Google's latest embedding model to encode documents and queries into 3072-dimensional space, removing local CPU/RAM overhead.
- Saves and queries vector representations via a local FAISS database, achieving milliseconds similarity search.
- Features a split frontend/backend setup: FastAPI running on AWS Elastic Beanstalk and Streamlit

running on Streamlit Cloud.

6. Technical Tech Stack

- Languages: Python 3.11
- APIs used: Google Gemini Embeddings API (gemini-embedding-2), Gemini LLM API (gemini-3.5-flash)
- RAG Frameworks: LangChain-Core, LangChain-Google-GenAI, FAISS-CPU
- Backend Deployment: AWS Elastic Beanstalk (Amazon Linux 2023, Python 3.11 platform, t3.micro free-tier)
- Frontend Deployment: Streamlit Cloud (Automatically updated via GitHub integration)
- Utilities: python-dotenv, FastAPI, Uvicorn, Requests

7. Cloud Deployment & Fixes

- Out-Of-Memory (OOM) Fix: Replaced massive PyTorch sentence-transformers with Google's hosted embedding API, reducing startup RAM utilization to under 90MB on AWS.
- Procfile formatting: Stripped CRLF carriage returns and trailing lines to ensure correct parsing by AWS Beanstalk platform engine.
- Port settings: Configured Procfile to bind Uvicorn to port 8000 to match AWS EB default Nginx proxy routing rules.
- Streamlit Secrets: Configured secrets fallback using st.secrets to seamlessly resolve dynamic API URLs in Streamlit Cloud environments.

8. Live Verification & Links

The project has been fully deployed. Verification tests show code 200 success with zero latency issues.

- Live UI Link: <https://ibmproject-gruy5zavcwsnugwcwx9js.streamlit.app/>

-	Live	Backend	REST	API	Endpoint:
http://Ai-email-copilot-env.eba-qtnai9na.us-east-1.elasticbeanstalk.com/generate-email					

- GitHub Repository: https://github.com/Sparsh2509/IBM_Project

9. Conclusion

The AI Email Copilot project successfully leverages AWS cloud capabilities and Google Gemini models to deliver a decoupled, scalable, and memory-efficient RAG system. The solution runs seamlessly within the free-tier constraints of both AWS and Streamlit Cloud, achieving corporate template compliance and natural drafts.